

Lab 4: VibeCoding — Fruit Catcher

Chat history with Claude (Claude Code session)

Shareable chat link	https://claude.ai/share/d3800e32-403a-4aaf-aeba-2ac41c783ceb
Repository	https://github.com/Its-Aditya-Singhal/64_fruit_catcher
Date	Wednesday, 7 October 2026
Tasks done	Task 1: floor-miss bug fix (c05d422) · Task 2: rotten fruit and bombs (b64ff6d) · Task 3: speed escalation (f46e152) · Task 4: splash particles (1a4b6f8)

This is a written copy of the conversation, in order. Messages are reproduced as sent. Where Claude ran commands or read files on the computer, a short grey note describes the activity instead of the raw tool output. Screenshots you sent are described in square brackets. The full original is at the shareable link above.

You · 11:45 (IST)

Okay analyze this folder whatever files are there in that folder and also it has one handout pdf of whatever I need to submit, all the deliverables and everything is there

[Folder attached: /Users/adityasinghal/Desktop/SElab4/64_fruit_catcher]

[Claude listed the folder, read README.md, main.py and the three files in game/, checked the git history, and read the handout PDF.]

Claude

I read the whole folder. It's a small Pygame game with one deliberate bug and three features to add, and the handout lists what to submit.

What's in the folder

- `main.py` (the entry point): opens a 600×500 window and runs the game loop at 60 FPS (frames per second).
- `game/game_engine.py` (the "brain"): holds the score, lives, the spawn timer and the game state (`PLAYING` or `GAME_OVER`). This is the file you'll edit most.
- `game/fruit.py`: a falling fruit with a random x position, a random speed of 4.0 to 6.5, and one of three colors.
- `game/basket.py`: the basket that moves left and right, kept inside the screen edges.
- `README.md`: the task list.
- `Lab_4_VibeCoding_Student_handout.pdf`: the lab rules.
- `.git`: already linked to github.com/Its-Aditya-Singhal/64_fruit_catcher, so it's your own copy, with 3 commits so far (all from the original repo).

The bug (Task 1)

In `GameEngine.update()`, the "fruit missed" branch is wrong:

```
if fruit.is_missed(self.height):
    self.score += 1          # wrong: rewards a miss
    self.fruits.remove(fruit)
```

- It should do `self.lives -= 1` instead of `self.score += 1`.
- Nothing ever sets `game_state = "GAME_OVER"`. When `lives <= 0`, you need to switch the state (the game's current mode), or the game can never end. The `GAME_OVER` screen and the R-to-restart code already exist but never run.

The three features

- **Task 2, rotten fruit and bombs:** add a new falling item type (for example a `Hazard` class in `fruit.py`, or an `is_bad` flag on `Fruit`). If the basket catches one, take a life or points away. Missing one on the floor should cost nothing.

- **Task 3, speed escalation:** the spawn delay is fixed at 750 ms and the speed is a fixed random range. Make the delay shrink (with a minimum) and the base speed grow as the score rises. `reset()` must also restore `spawn_delay`, or restarting would keep the hard settings.
- **Task 4, splash particles:** add a small `Particle` class, with a list in the engine. Spawn droplets in the fruit's color when it's caught or hits the floor. Update them each frame (move, fade, remove when done) and draw them.

What to submit

The README and the handout don't quite agree, so check which one your instructor wants:

- **Handout:** put everything in a `Lab-4` folder in your repo, with the before video, the after video, the updated code and the chat history as a doc or PDF. Make a separate commit for each task. Don't raise a PR (pull request) to the main SETAPESU26 repo. Work individually.
- **README checklist:** a 10-second video before the changes, a 10-second video after, and the link to your LLM chat with the full history.

Suggested order

1. Run `python main.py` and record the 10-second "before" clip. Let a fruit hit the floor so the score goes up and lives stay at 3.
2. Do Tasks 1 to 4 one at a time, each as its own LLM prompt and its own commit. The handout wants each one done in 3 to 4 prompts.
3. Record the "after" clip showing a life lost, a bomb, faster fruit, splashes, and Game Over followed by R to restart.
4. Export the chat, put everything in `Lab-4`, and push.

I can make the code changes for Tasks 1 to 4 directly in the folder, as four separate commits. One catch: the handout wants the work done through your own LLM chat, and the chat history is part of what you submit. If you want that history to be clean, do the prompting in a separate chat and use me only to review the result. Which would you like?

You · 11:48 (IST)

we need to follow what handout is telling us to do

Claude

Understood, we'll follow the handout. Here is what it requires, and who does each part.

Handout rules

1. Work individually, using one vibe coding tool (an AI that writes code from your prompts). Aim for 3 to 4 prompts per task.
2. Clone or fork the repo. Yours is already set up at `Its-Aditya-Singhal/64_fruit_catcher`.
3. Record 10 seconds of the game before any changes.
4. Fix the bug, then add the features one by one.
5. Make a separate commit (a saved checkpoint in git) for each task, so 4 commits.
6. Record 10 seconds after all changes.
7. Push to your own repo, with no PR (pull request) to SETAPESU26.
8. Put everything in a `Lab-4 folder` in the repo: the before and after videos, the updated code, and the chat history as a doc or PDF.

Who does what

- **You:** record both videos (I can't record your screen), export the chat, and run the final `git push` with your GitHub login.
- **Me, if you want to use this chat as your vibe coding tool:** do Tasks 1 to 4 in order, with one commit per task and a clear commit message. I'd also test that the game runs and set up the `Lab-4` folder layout. This chat would then be the chat history you export.

One decision for you: should I make the changes here, with this conversation as your chat history? If you'd rather use ChatGPT or Gemini for the prompting, I'll stay out of the code and only check the result.

If it's me, record the "before" video first, since once I fix the bug it won't show the broken behavior anymore. Tell me when that's done and I'll start with Task 1.

You · 11:56 (IST)

[Screenshot: IDLE shell, Python 3.13.0 on darwin, running /Users/adityasinghal/Desktop/SElab4/64_fruit_catcher/main.py, ending in "ModuleNotFoundError: No module named 'pygame'"]

some error occurred and its working, tell how to fix thios

Claude

This error means pygame (the game library) isn't installed for the Python you're using. It isn't a problem with the code.

You ran it from IDLE (Python's built-in editor) with Python 3.13, which doesn't have pygame yet.

Fix

1. Open **Terminal** (press Cmd+Space, type "Terminal", press Enter).
2. Install pygame for that same Python version:

```
python3.13 -m pip install pygame
```

Using `python3.13 -m pip` makes sure it goes into the same Python that IDLE uses. A plain `pip install pygame` can install it for a different Python.

3. Run the game from Terminal:

```
cd ~/Desktop/SElab4/64_fruit_catcher
python3.13 main.py
```

Run it from Terminal rather than IDLE, because pygame windows sometimes misbehave inside IDLE on Mac.

If step 2 fails

- `command not found: python3.13`: try `python3 -m pip install pygame`, and check the version with `python3 --version`.
- An "externally managed environment" error: make a virtual environment (an isolated Python setup just for this project):

```
cd ~/Desktop/SElab4/64_fruit_catcher
python3 -m venv venv
source venv/bin/activate
pip install pygame
python main.py
```

When the game window opens, let a few fruits hit the floor. The score should go up while lives stay at 3, and that's your "before" recording. Tell me when you've recorded it and I'll start on Task 1.

You · 12:01 (IST)

I am not able to play or interactive in that game windows when i run main.py
is it the issue??

Claude

Yes, that's a problem, but it's a known one on Mac and not a bug in the code. Try these in order.

1. Click the game window once.

On macOS the window often opens without keyboard focus (it isn't "listening" to your keys yet), so A/D and the arrow keys do nothing. Click inside the window, then press the keys. If it opened behind other windows, bring it to the front first.

2. Run it from Terminal, not IDLE.

IDLE and pygame often clash on Mac, which gives a frozen or unresponsive window. Use:

```
cd ~/Desktop/SElab4/64_fruit_catcher
python3.13 main.py
```

Make sure no IDLE copy is still running (quit IDLE completely).

3. Check that the game is actually running.

Fruits fall on their own, with no keys needed. Check which of these you see:

- **Fruits falling, but the basket won't move:** it's a focus or keyboard problem, so go to step 4.
- **Nothing falling, or the window is frozen and shows "not responding":** the game loop isn't running properly. Close it, run it from Terminal as in step 2, and tell me any error text.

4. Give Terminal keyboard permission.

Open System Settings → Privacy & Security → Input Monitoring, and turn on **Terminal** (or IDLE or Python if that's what you launch from). Quit and reopen Terminal, then run it again.

5. Upgrade pygame.

An old pygame can misbehave on newer macOS:

```
python3.13 -m pip install --upgrade pygame
```

Tell me which of these you see, with any text from Terminal, and I can narrow it down. If fruits are falling but the basket won't move, it's almost surely step 1 or step 4.

You · 12:05 (IST)

done I've recorded the before video now next step?

[Claude edited game/game_engine.py on the user's computer (keeping the file's Windows line endings), then ran a headless test. A first attempt to commit failed because git on that shell had no user name or email set, so Claude left the change staged for the user to commit.]

Claude

Task 1 is fixed and tested, but I couldn't make the commit. My shell on your Mac has no git name or email set, and I didn't want to put a made-up identity on your commits. The change is already staged (added to git, ready to commit).

What changed: in `game/game_engine.py`, a fruit hitting the floor now costs a life instead of adding +1 score. At 0 lives the game goes to Game Over, and R restarts it. I tested this with a headless run (the game running without a window): lives went 2, 1, 0, then Game Over, and the score stayed at 0. Reset restored 3 lives.

Commit it from your Terminal so it uses your own GitHub identity:

```
cd ~/Desktop/SElab4/64_fruit_catcher
git commit -m "Task 1: Fix floor miss to deduct a life and trigger Game Over"
```

If git asks who you are, run these once and then commit again:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

You can try the fix yourself with `python3.13 main.py`. Let three fruits hit the floor and you should see Game Over.

Tell me when the commit is done and I'll start Task 2 (rotten fruit and bombs). For the same identity reason, I'll leave each later commit to you as well.

You · 12:12 (IST)

now add one feature -rotten fruit and bombs

[Claude rewrote game/fruit.py (new item kinds and drawing) and edited the catch/miss logic in game/game_engine.py, then ran a headless test of every catch/miss case and the spawn mix.]

Claude

Task 2 is added and tested, and the code is ready for you to commit.

What I added

- **Rotten fruit** is a sickly green with dark spots. Catching one takes 2 points off your score, and the score never goes below 0. Letting it fall costs nothing.
- **Bombs** are black with a fuse and a spark. Catching one costs a life, and at 0 lives it's Game Over. Letting one fall costs nothing.
- Good fruit works as before: +1 when caught, and -1 life when it hits the floor.
- Each new item is about 70% good fruit, 15% rotten and 15% bomb.
- The changes are in `game/fruit.py` (a `kind` field and the new drawing) and the catch and miss logic in `game/game_engine.py`.

Test results (headless, with no window): the score and lives changed exactly as above for every case. A bomb at 1 life gives Game Over, the spawn mix came out at 69/15/16%, and drawing the new items works.

Your commits. I still haven't committed Task 1, so please make the two commits in order:

```
cd ~/Desktop/SElab4/64_fruit_catcher
git commit -m "Task 1: Fix floor miss to deduct a life and trigger Game Over"
git add game/fruit.py game/game_engine.py
git commit -m "Task 2: Add rotten fruit and bomb hazards"
```

Task 1 is already staged, and the first command saves it. The second command then stages and commits Task 2 as its own checkpoint. If git asks who you are, set your name and email as in my last message.

Run `python3.13 main.py` to see it for yourself. Tell me when you've committed and I'll start Task 3 (speed escalation).

You • 12:13 (IST)

I already committed task 1 with the commands you gave before

[Claude checked the git log: Task 1 was committed as c05d422.]

Claude

Good, Task 1 is committed (`c05d422`). So my last message had one step too many. Only Task 2 is left to commit, and it shows as modified in `game/fruit.py` and `game/game_engine.py`. Run just this:

```
cd ~/Desktop/SElab4/64_fruit_catcher
git add game/fruit.py game/game_engine.py
git commit -m "Task 2: Add rotten fruit and bomb hazards"
```

When that's done, say so and I'll start Task 3 (speed escalation). If you want, I can start writing it now and you commit Task 2 before testing it. The two changes would sit together in the same file, so committing Task 2 first keeps the commits separate and clean.

You • 12:14 (IST)

[Screenshot: Terminal showing the Task 1 commit succeeded as c05d422, then "fatal: Unable to create '.git/index.lock': File exists." when running git add / git commit for Task 2]

error

Claude

That's a leftover lock file from my earlier git commands. They couldn't clean up after themselves because my shell isn't allowed to delete files in your folder. The lock is stale (leftover, nothing is actually running), so it's safe to remove.

Run this in your Terminal:

```
cd ~/Desktop/SElab4/64_fruit_catcher
rm .git/index.lock
git add game/fruit.py game/game_engine.py
git commit -m "Task 2: Add rotten fruit and bomb hazards"
git log --oneline -3
```

The last command should list the Task 2 commit on top of the Task 1 commit (c05d422). If `rm` says the file doesn't exist, skip it and run the rest.

To avoid this again, I'll make no more git commands in your folder. I'll only edit code files, and you do all the `git add` and `git commit` steps.

Once the log looks right, tell me and I'll do Task 3 (speed escalation).

You • 12:15 (IST)

[Screenshot: Terminal showing "[main b64ff6d] Task 2: Add rotten fruit and bomb hazards" and git log with b64ff6d (HEAD -> main), c05d422, 74e9f7b (origin/main)]

done?

[Claude edited game/fruit.py and game/game_engine.py for Task 3, printed the diff, and ran a headless test of the difficulty curve and reset.]

Claude

Yes, both commits are in. The log shows Task 2 (b64ff6d) on top of Task 1 (c05d422). Now Task 3, the speed escalation.

Task 3 is added and tested. The code changes are ready for you to commit.

How it works. Every 5 points is a new level, and each level makes new items harder:

- The spawn delay (the gap between items) shrinks by 50 ms per level, from 750 ms down to a floor of 250 ms.
- Newly spawned items fall 0.5 faster per level, capped at +5.0 so the game stays playable.
- Restarting with R resets the level, speed and delay to the starting values.
- A **Level** label now shows at the top center of the screen, which helps in your "after" video.

Test results (headless, with no window): at score 0 the delay was 750 ms and speeds were 4.0–6.5. At score 20 (level 5) it was 550 ms and 6.0–8.5, and at score 40 (level 9) it was 350 ms and 8.0–10.5. Score 100 hit the caps (250 ms and +5.0 speed). Reset brought everything back to the start values.

Commit it in your Terminal:

```
cd ~/Desktop/SElab4/64_fruit_catcher
git add game/fruit.py game/game_engine.py
git commit -m "Task 3: Add dynamic falling speed escalation"
```

Tell me when it's done and I'll start Task 4 (splash particles).

You • 12:16 (IST)

[Screenshot: GitHub "Commits" page for Its-Aditya-Singhal/64_fruit_catcher on main, showing only the three original commits (Update game_engine.py, Update README.md, Add files via upload)]

where is task 2 commit?

Claude

The GitHub page only shows the three original commits because **you haven't pushed yet**. Your Task 1 and Task 2 commits exist only on your Mac. They went into your local git history (the Terminal log showed b64ff6d and c05d422), and GitHub only learns about them after a `git push`.

I'll do the push at the end of the lab, after all four tasks are committed. You can also push now, if you want to see them on GitHub:

```
cd ~/Desktop/SElab4/64_fruit_catcher
git push origin main
```

After that, refresh the Commits page and Tasks 1 and 2 should appear. The push can ask you to log in to GitHub. Pushing early doesn't affect anything else.

One thing is still missing before Task 3: I haven't seen a commit for it yet. Run this when you're ready:

```
git add game/fruit.py game/game_engine.py
git commit -m "Task 3: Add dynamic falling speed escalation"
```

Then I'll start Task 4 (splash particles).

You · 12:17 (IST)

```
cd ~/Desktop/SElab4/64_fruit_catcher
git add game/fruit.py game/game_engine.py
git commit -m "Task 3: Add dynamic falling speed escalation"
```

okay I ran these commands and committed task 3

[Claude confirmed Task 3 as f46e152 in the git log, then created game/particle.py and edited game/game_engine.py for Task 4, and ran a headless test of the particle logic.]

Claude

Task 4 is added and tested, and the code is ready for you to commit.

What it does

- A new file, `game/particle.py`, holds a small `Particle` class. Each droplet shoots upward, then gravity pulls it down while it shrinks and disappears after about 0.4 to 0.7 seconds.
- Each catch or floor hit releases 14 droplets in the item's own color. Bombs burst into orange, red and grey sparks instead.
- A catch splashes at the basket, and a floor hit splashes on the ground line.
- Droplets keep animating on the Game Over screen. Pressing R clears them.
- The changes are in `game/game_engine.py`, plus the new `game/particle.py`.

Test results (headless, with no window): a catch gave 14 droplets, and a floor hit and a bomb each added 14 more. All were gone after about 68 frames. Reset cleared them, and Game Over drew without errors. I couldn't check how it looks on screen from my side, so you'll see that when you run it.

Commit it in your Terminal. This time there's a new file, so add all three:

```
cd ~/Desktop/SElab4/64_fruit_catcher
git add game/particle.py game/game_engine.py
git commit -m "Task 4: Add fruit splash particle effects"
git log --oneline -5
```

After that, only the final steps remain:

1. Run `python3.13 main.py` and record the **after video**. Try to show a catch splash, a floor splash, a bomb, a rotten fruit, the Level number rising, Game Over, and R to restart.
2. Make the `Lab-4` folder with the two videos, the code, and the exported chat. The handout says the chat history goes in as a doc or PDF.
3. `git push origin main`.

Tell me when Task 4 is committed. If you'd like help setting up the `Lab-4` folder and its README, I can do that next.

You · 12:18 (IST)

done committed

[Claude checked the git log (read-only): all four task commits present.]

Claude

All four task commits are in, one per task:

```
1a4b6f8 Task 4: Add fruit splash particle effects
f46e152 Task 3: Add dynamic falling speed escalation
b64ff6d Task 2: Add rotten fruit and bomb hazards
c05d422 Task 1: Fix floor miss to deduct a life and trigger Game Over
```

The code is done. These steps are left, and I can't do any of them for you:

1. Record the "after" video (10 seconds). Run `python3.13 main.py` and try to show these:

- a fruit catch and a floor miss, with the splash droplets
- a bomb costing a life, and a rotten fruit costing score
- the **Level** number going up (fruit gets faster as it climbs)
- Game Over, then R to restart

2. Make the Lab-4 folder (the handout wants it inside your repo):

```
cd ~/Desktop/SElab4/64_fruit_catcher
mkdir Lab-4
```

Move your before video and after video into it. Then export this chat as a PDF or doc (the chat history is a required deliverable) and put it there too.

The handout also lists "updated code" as a deliverable. The code is already in the repo, so that part is covered. If you want a copy inside Lab-4 as well, copy `main.py` and the `game` folder in.

3. Commit and push:

```
git add Lab-4
git commit -m "Add Lab-4 deliverables: before/after videos and chat history"
git push origin main
```

Check that the video files aren't too big for GitHub. Each file must be under 100 MB, and a 10-second clip should be fine. Don't open a PR to SETAPESU26, as the handout says.

4. Check on GitHub: refresh the Commits page, and you should see all four task commits plus the final one. Open the Lab-4 folder there and confirm the videos play.

The handout wants this to be your own work, so read through the code I wrote before you submit it. You may be asked to explain it. If you'd like, I can walk you through each change, or write a short summary of the four tasks for your submission.

You · 12:23 (IST)

its showing the videos are QT movie not an mp4 file, is it fine or should I convert it and if yes how?

Claude

The `.mov` format is fine. The handout only asks for a before and an after video and doesn't name a format. A Mac screen recording is a standard H.264 video in a QuickTime container, and GitHub can play `.mov` files in the browser.

Converting only makes sense if one of these applies:

- **The file is over 100 MB.** GitHub rejects files that big. A 10-second clip is normally only a few MB, so check the size in Finder.
- **Your instructor says they need `.mp4`.**
- **You want it to play for people on Windows.** Windows can have trouble with `.mov`.

Don't just rename `.mov` to `.mp4`. It sometimes appears to work, but it's not a real conversion and can break playback.

To convert it:

1. ffmpeg (a command-line video tool), if you have it. Check with `ffmpeg -version`. Then:

```
cd ~/Desktop/SElab4/64_fruit_catcher/Lab-4
ffmpeg -i before.mov -c:v libx264 -c:a aac before.mp4
ffmpeg -i after.mov -c:v libx264 -c:a aac after.mp4
```

Use your real file names in place of `before.mov` and `after.mov`. If you don't have `ffmpeg`, `brew install ffmpeg` installs it (if you use Homebrew).

2. **QuickTime Player, with no install.** Open the video, choose File → Export As → 1080p (or 720p), and save it. The result is an `.mp4`-style file saved as `.m4v`. Rename it to `.mp4`, since an H.264 `.m4v` plays fine under that name.

3. **Leave the `.mov` files as they are.** This is fine if they're small and your instructor isn't picky.

I'd leave the `.mov` files unless you've checked a size problem or been told to use `.mp4`. If you want to convert but don't want to install anything, put the files in the `Lab-4` folder and tell me their names. I'll check whether I can convert them here.

You · 12:26 (IST)

how can I download this chat history as a pdf along with a sharable link of this chat

[Claude searched Claude's support documentation and Claude Code docs for sharing and export instructions.]

Claude

I can't see your app's menus, so I don't know exactly where the buttons are. These are the options, depending on what your app shows.

Shareable link

- Look for a **Share** button or menu on this conversation (usually near the title at the top). Claude's help center confirms that shared chats are managed from a "Share" menu, and that you can turn a link off later under Settings → Privacy → Shared chats (Public links for shared chats: <https://support.claude.com/en/articles/16762437-public-links-for-shared-chats>). The step-by-step guide is in Share and unshare chats (<https://support.claude.com/en/articles/10593882-share-and-unshare-chats>).
- If there's no Share option in this kind of session, tell me what you see in the app and I'll help you find the right one.
- Open the link in a private or incognito window to check that someone else can see it. The handout wants a link to the full chat history, so a link that only works for you won't help.

PDF of the chat

- The simplest way is to open the chat in a browser (the shared link works), then use the browser's Print → **Save as PDF**. In Safari that's File → Export as PDF, or Print → the PDF menu at the bottom left.
- The Claude Code docs describe an `/export` command, but it saves plain text from a terminal session, and I can't confirm it's in your app. Check by typing `/export` in the message box.

If neither works: I can write a text transcript of this conversation myself and save it as a PDF in your `Lab-4` folder. The catch is that it would be my version of the chat, built from this session, with long tool output trimmed. A real export or share link is more convincing as the "complete chat history", so use mine only as a backup.

Do you see a Share button? If not, tell me what the top of the chat window looks like.

You · 12:28 (IST)

okay generate pdf for me of this whole chat along with this sharable link I am providing you, add this link also in the pdf

<https://claude.ai/share/d3800e32-403a-4aaf-aeba-2ac41c783ceb>

[Claude is generating this PDF. (This is the last entry: Claude's closing message about the finished PDF is not included, because the PDF is created before that message is written.)]